Building a Custom GraphRAG System for History Class Transcripts: Architecture, Research, and Implementation Guide

A consolidated research document covering ontology design, extraction pipelines, retrieval architecture, and production deployment for a Neo4j-based conversational knowledge graph.

Executive Summary

This document consolidates research across schema design, extraction pipelines, retrieval architecture, community detection, and production deployment for building a custom GraphRAG system processing English history class transcriptions (Predictive History, ~134 episodes, ~125 hours) on Neo4j. The findings draw on **90+ references from 2024–2026** across six architectural domains.

Core thesis: A custom-built GraphRAG pipeline—using direct OpenAI API calls, Neo4j, and a domain-specific ontology—will dramatically outperform any pre-built framework for a conversational class corpus. Schema-constrained extraction achieves **98.8% precision** at scale versus open extraction's **40% predicate error rate**, and conversation-specific challenges like speaker attribution, cross-turn assertions, and claim modeling are not addressed by any existing framework.

Key findings across all research domains:

- The LoCoMo benchmark validates that **transforming dialogue into speaker specific assertions dramatically improves RAG performance** (87.9% human F1 vs. 37.8% for GPT-3.5)
- Kelsey Rook's ISWC 2025 formalization of Open Knowledge Extraction from Dialogue (OKE-D) establishes the theoretical foundation for dialogue-to-KG pipelines
- KGGen demonstrates an **18% improvement** over GraphRAG at NeurIPS 2025 via multi-stage extraction
- LazyGraphRAG dominates with **95/96 statistical wins** at 0.1% of full GraphRAG indexing cost
- Multi-pass extraction is strictly superior: ECR shows **+16% improvement in triple F1**, CORE-KG proves removing coreference resolution causes a **28.32% increase in node duplication**
- SAC-KG achieves **89.32% precision** at 1M+ node scale, a **>20% increase over SOTA**
- No single framework addresses the intersection of temporal modeling, speaker attribution, and Portuguese-language conversational data

1. Why Custom-Built Pipelines Produce Superior Knowledge Graphs

The case against pre-built frameworks for domain-specific conversational data rests on three pillars: generic extraction defaults, conversation-blind architectures, and prohibitive costs at scale.

1.1 Framework-specific limitations

Microsoft GraphRAG's default extraction prompt recognizes only four entity types— Person,

Organization, Geography, and Event—designed originally for news articles. The GitHub repository warns: "Using GraphRAG with your data out of the box may not yield the best possible results." For a series of live classes on History, relevant entities include monetary policy stances, geopolitical predictions, ideological opinions, and macro narratives—none of which map to these four types. Additional limitations include **~610,000 tokens per global query** (versus ~100 for LightRAG—a 6,000× difference), static summaries requiring full reindexing, English-default indexing even for non English sources, and version instability with breaking changes. Shereshevsky reports that indexing costs reach **$33K for large datasets**—untenable for a 300-episode corpus needing incremental updates.

LightRAG performs open extraction with zero schema constraints, producing inconsistent relationship types and entity duplication. Its own paper acknowledges a "reduced depth in examining specific entities" as a trade-off. The framework was withdrawn from ICLR 2025 review. In practice, open extraction without ontology guidance produces **40% predicate errors** according to the Plumber framework analysis.

LangChain adds **~10ms overhead per call** and ~2,400 tokens per query versus ~1,570 for Haystack (AIMultiple benchmark, 2025). **LlamaIndex's PropertyGraphIndex** provides modular extractors and retrievers with Neo4j backend support but lacks conversation-specific optimizations.

1.2 Quantitative evidence for schema-constrained extraction The

strongest evidence comes from multiple 2024-2025 studies:

- The VLDB 2024 workshop paper on ontology-guided Chain-of-Thought prompting showed that **ontology conformance markedly increased** across all 10 tested domains when extraction was schema-constrained
- ODKE+ demonstrated **98.8% precision across 195 predicates** processing 9 million Wikipedia pages using ontology-guided prompting
- The ACL TextGraphs 2024 paper "Prompt Me One More Time" confirmed that two stage ontology-verified extraction "increases performance in KG construction tasks"
- The comprehensive survey by Bian et al. (arXiv:2510.20345, October 2025) documents the evolution from rigid ontology-driven methods through adaptive embedding-based integration to Schema-Constrained Dynamic Decoding, finding **schema-based methods achieve high consistency but limited flexibility; schema-free methods show LLMs can internalize latent relational structures through guided reasoning**
- Kommineni et al. (arXiv:2403.08345, 2024) demonstrate the "schema-first" paradigm where LLMs generate competency questions, construct ontology, then populate under schema supervision

The practitioner star-graph pattern documented by Partha Sarkar demonstrates that **focused domain-specific extraction reduces complexity by 90%** (19 relationships vs. 190 for 20 entities).

1.3 Conversational data presents unique structural challenges

Kelsey Rook's ISWC 2025 Doctoral Consortium paper ("Open Knowledge Extraction from

Dialogue Using In-Context Learning," CEUR Vol-4085) is the single most relevant academic reference. It formalizes **OKE-D (Open Knowledge Extraction from Dialogue)** and identifies four key challenges:

1. **Cross-turn assertions** with higher subject-object span distances than documents 2. **Anaphora and pronoun density** requiring robust coreference 3. **Dialogue acts** where not all utterances assert facts (some are opinions, suggestions, or hedges)
4. **Multiple perspectives** where speakers express conflicting views

Rook proposes a **perspective-aware dialogue ontology** integrating speaker identity, temporal context, and dialogue acts using SIO and PROV-O. The paper demonstrates that even frontier models (Gemini 2.5 Pro) fail to extract the *important* knowledge from dialogues without specialized prompting. The chunking strategy uses a sliding window of n utterances calibrated to subject-object span distance distributions.
The **LoCoMo benchmark** (Maharana et al., ACL 2024; arXiv:2402.17753) evaluates very long-term conversational memory across ~300 turns and up to 35 sessions. Key findings: human F1 reaches **87.9%** while GPT-3.5-16K achieves only **37.8%** and the best RAG achieves **41.4%**—LLMs lag **56% behind humans**. Temporal reasoning shows a **73% gap** behind human levels. The most critical finding: **"RAG does particularly well when dialogues are transformed into a database of assertions (observations) about each speaker's life and persona"**—directly validating the knowledge graph extraction approach.

1.4 The GraphRAG framework landscape

LazyGraphRAG (Edge et al., Microsoft Research, November 2024) eliminates upfront LLM summarization, with indexing cost at **0.1% of full GraphRAG**. It won **95 of 96 comparisons** against Vector RAG, GraphRAG Local/Global/DRIFT, LightRAG, RAPTOR, and TREX with statistical significance. **LightRAG** (Guo et al., HKU, 2024) uses **~100 tokens per query** versus GraphRAG's ~610,000 (**6,000× saving**) with simple incremental updates via graph union. **GraphRAG-Bench** (Xiang et al., ICLR 2026) provides the first rigorous cross-framework benchmark, finding that GraphRAG **frequently underperforms vanilla RAG on many real-world tasks**, with guidelines for when graph structure provides measurable benefits.

Three comprehensive surveys from 2024–2025 map the space: Peng et al. (ACM TOIS 2025), Han et al. (arXiv:2501.00309), and Zhang et al. (arXiv:2501.13958, associated with ICLR 2026). The **Awesome-GraphRAG** repository (DEEP-PolyU, continuously updated) tracks emerging systems including PathRAG, ArchRAG, SubgraphRAG, LinearRAG, and FastGraphRAG.

1.5 Decision criteria

Lanham (Medium, February 2026) provides a practical decision framework: Pipeline RAG for single-hop factual queries, Agentic RAG for multi-part queries requiring iteration, Knowledge Graph RAG for relational queries and global synthesis. GraphRAG approaches outperform baseline RAG in **70–80% of complex sensemaking tasks** (Microsoft Research).

Property graphs are the right choice. ISO/IEC 39075:2024 (GQL) is the first new ISO database language standard since SQL in 1987, eliminating the historical standardization advantage of RDF/SPARQL. Angles et al. (ACM SIGMOD, December 2024) demonstrate existing RDF→property

graph transformations lose **up to 70% of query information**. Neo4j's comparison blog (June 2024) notes RDF requires additional triples/blank nodes for relationship properties, ontology-driven approaches face expensive skill building, and SPARQL query termination is not guaranteed. TigerGraph's decision framework (2025) recommends property graphs for analytics, multi-hop reasoning, real-time traversal, AI integration, and hybrid search/RAG. DataVera (July 2025) notes "OWL ontologies are not often used in GraphRAG solutions" due to complexity.
The recommendation: use Microsoft GraphRAG or LightRAG for static document corpora; use Graphiti for conversation memory with temporal tracking; **build a custom pipeline when you need fine-grained control over conversational graph schemas, speaker attribution, bi-temporal modeling, and domain-specific retrieval strategies—exactly the Predictive History use case.**

2. Knowledge Graph Schema Design for Conversational Data

2.1 The utterance-as-node pattern: evidence and implementations

The core design pattern—making each utterance or speaker turn a first-class node in a property graph—has gained strong empirical support across at least five distinct real world systems with published schemas.

SGMem (Wu et al., arXiv:2509.21212, September 2025) represents dialogue as sentence-level graphs within chunked units, capturing turn-, round-, and session-level associations. Sentences serve as nodes with edges encoding temporal and semantic relationships, outperforming baselines on both LongMemEval and LoCoMo benchmarks.

A critical January 2026 paper, **"Does Memory Need Graphs?"** (Hu et al., arXiv:2601.01280), introduces a unified framework decomposing dialog memory systems into core components. Its key finding: sentences-as-nodes can outperform entity-level KG approaches for conversational memory, providing design-choice guidance across memory representation, organization, maintenance, and retrieval.

MAGMA (Jiang et al., arXiv:2601.03236, January 2026) extends this with orthogonal semantic, temporal, causal, and entity graphs per memory item, achieving **45.5% higher reasoning accuracy** on long-context benchmarks via policy-guided traversal.

Jokinen & Wilcock (IWSDS 2025) propose a four-layer dialogue history graph: Dialogue → Turns → Semantic Units → Conversational Entities—directly mapping to the Document→Chunk→Entity hierarchy needed for podcast transcripts. The **Grounded Graph (G²)** model (Bikaki et al., Knowledge-Based Systems, 2024) unifies dialogue utterances and knowledge passages as graph nodes with semantic edges, demonstrating superiority over sequence-based approaches. **D-HGN** constructs heterogeneous dialogue graphs with three node types (utterance, speaker, knowledge) connected via "spoken-by" and "relates-to-knowledge" edges.

Two comprehensive surveys track this landscape: **"Memory in the Age of AI Agents"** (Hu et al., arXiv:2512.13564, December 2025) covers graph-based memory systems including Graphiti, SGMem, MAGMA, and Mem0. **"Graph-based Agent Memory: Taxonomy, Techniques, and Applications"** (Yang et al., arXiv:2602.05665, February 2026) provides a taxonomy covering episodic, semantic, and procedural memory modeled as graphs.

Production implementations

Di Bratto et al. (2021) built a Neo4j graph for movie recommendation dialogues with **17 million nodes and 75 million relationships**. Their schema uses UTTERANCE nodes (carrying text and turn number) connected via FOLLOWED_BY edges to form a linked-list chain. Speaker identity is encoded as additional node labels (:UTTERANCE:RECOMMENDER or :UTTERANCE:SEEKER), and entity grounding happens through REFERS_TO relationships to domain entities. This creates "ring patterns"—when distant utterances reference the same entity, the graph forms cycles that cross between the dialogue chain and the domain knowledge graph, enabling disambiguation that flat text cannot achieve.

```
cypher

// Di Bratto chain + entity star pattern
CREATE (u1:UTTERANCE:RECOMMENDER {text: "Have you seen Knives Out?", turn: 1}) CREATE
(u2:UTTERANCE:SEEKER {text: "No, who's in it?", turn: 2})
CREATE (u1)-[:FOLLOWED_BY]->(u2)
CREATE (u1)-[:REFERS_TO]->(m:MOVIE {name: "Knives Out"})
CREATE (u3:UTTERANCE:RECOMMENDER {text: "Chris Evans is great in it", turn: 3}) CREATE
(u3)-[:REFERS_TO]->(p:PERSON {name: "Chris Evans"})
CREATE (p)-[:WORKED_IN]->(m)
```

DialogueGCN (EMNLP 2019, widely cited) models each utterance as a graph node with two edge types: self-dependency edges connecting a speaker's own utterances, and inter-speaker dependency edges capturing cross-party influence. While this is a computational graph for emotion recognition rather than a database schema, its
speaker-conditioned edge types have influenced subsequent conversation graph designs.

GraphWOZ (2022) models dialogue state itself as a dynamically-updated knowledge graph. Entity types include Person , Location , Event , and Group , with mention nodes linking utterance text spans to knowledge graph entities. Each user turn adds or modifies nodes and relations.

Common utterance node properties across implementations: uuid / id , text / content , embedding (vector), timestamp / created_at , speaker / role , and session_id / group_id . Sentiment and intent labels appear in annotated datasets but are less common in production systems.

2.2 Graphiti/Zep: the production standard for conversation knowledge graphs

Graphiti (Rasmussen et al., arXiv:2501.13956, January 2025; 22,800+ GitHub stars, Apache 2.0 license, 25,000+ weekly PyPI downloads) represents the most mature production system. It defines the graph as **G = (N, E, φ)** with three hierarchical subgraphs.

The **Episode Subgraph** preserves raw conversation data. Each message becomes an Episodic node storing verbatim content, speaker/actor identity, reference timestamp, and source type (message/text/json). Episodes chain via NEXT_EPISODE edges within a group_id . This layer is non-lossy—the ground truth corpus is always recoverable.

The **Entity Subgraph** captures extracted semantic knowledge. Entity nodes carry name , summary (LLM-generated, updated as new information arrives), and name_embedding (1024-dimensional vector). The critical innovation is on the edges:

each entity-to-entity relationship stores a fact property in natural language plus a fact_embedding for semantic search. Episodic edges connect entities back to their source episodes, enabling full provenance tracing.

The **Community Subgraph** provides high-level topic summaries. Label propagation clusters strongly-connected entities into Community nodes with name (key terms, embedded for search) and summary (iterative map-reduce summarization).

Bi-temporal edge properties make Graphiti unique. Every semantic edge carries four timestamps:

Property	Timeline	Purpose
created_at	Transaction (T')	When fact was ingested into the system
expired_at	Transaction (T')	When fact was invalidated in the system
valid_at	Event (T)	When fact became true in reality
invalid_at	Event (T)	When fact stopped being true

When contradictory information arrives, the old edge's invalid_at is set while a new edge is created. Old edges are never deleted—the fact text is regenerated to reflect history (e.g., "Maria used to work as junior manager, until her promotion to senior manager"). This handles non-chronological episode ingestion: if episodes arrive out of order, valid_at dates maintain correct world-timeline ordering.

Graphiti achieves **94.8% on Deep Memory Retrieval** versus 93.4% for MemGPT; Zep outperforms Mem0 by 10% on Mem0's own benchmark.

2.3 The lexical graph pattern

Neo4j's **lexical graph pattern**, formalized in the neo4j-graphrag-python library, provides the canonical architecture for connecting text chunks to both their document hierarchy and extracted entities. Replace Document with Conversation/Episode , Chunk with Utterance/Segment , and the pattern transfers directly.

```python
# From neo4j-graphrag-python LexicalGraphConfig
document_node_label = "Document" # Container
chunk_node_label = "Chunk" # Text unit with embedding
chunk_to_document_relationship = "FROM_DOCUMENT" # Hierarchy
next_chunk_relationship = "NEXT_CHUNK" # Sequential chain
node_to_chunk_relationship = "FROM_CHUNK" # Entity provenance
```

Chunk nodes carry id , index (position), text , and embedding (vector). **Entity nodes** use a base label __Entity__ plus domain-specific labels with name , description , and embedding properties. The NEXT_CHUNK chain preserves document order while FROM_CHUNK relationships provide entity-to-source provenance.

This creates a **dual-path retrieval architecture**: the vector RAG path searches chunk embeddings

for semantic similarity; the graph RAG path starts from vector-matched chunks, traverses FROM_CHUNK edges to entities, then follows entity-to-entity relationships for multi-hop reasoning.

Feature	Neo4j LLM Graph Builder	Microsoft GraphRAG	LlamaIndex PropertyGraph
Doc→Chunk link	FROM_DOCUMENT	document_id field	Node metadata
Chunk ordering	NEXT_CHUNK chain	Sequential index	Implicit
Chunk→Entity	FROM_CHUNK	text_unit_ids array	Optional
Community detection	Leiden (GDS)	Leiden hierarchical	Optional
Vector embeddings	Chunk + Entity	Entity descriptions	Entity + Chunk

The **Apollo 11 transcript case study** (Showpad Engineering blog) applied Microsoft GraphRAG to mission voice transcripts, extracting **933 nodes and 2,226 relationships**. The graph successfully answered questions that naive RAG failed—like identifying that "HORNET" was the USS Hornet recovery vessel—because the graph explicitly modeled entity roles and dependencies across dialogue.

2.4 A conversation-centric ontology for the Market Makers podcast

The schema must reflect the conversational DNA of a podcast: the atomic unit is not a document paragraph but a **speaker utterance**, and the most valuable analytical entities are not just people and organizations but **claims, predictions, and opinions** attributed to specific speakers at specific moments in time.

Core node types

The ontology uses seven primary node types organized in three layers:

Content layer — Episode, Segment, and Utterance nodes form the structural backbone. An Episode contains 8-12 Segments (coherent discussion blocks of ~10-15 minutes), each containing multiple Utterances (individual speaker turns). This three-tier hierarchy enables both coarse retrieval (episode-level) and precise attribution (utterance-level).

```cypher
CREATE (e:Episode {
 episodeId: "EP-2024-0315", title: "O futuro da Selic e o cenário fiscal",
 publishDate: date('2024-03-15'), duration: duration('PT2H05M'), episodeNumber: 245 })
CREATE (s:Segment {
 segmentId: "seg-245-01", startTime: duration('PT0M'), endTime: duration('PT25M'),  summary:
"Discussion on Selic trajectory and COPOM signaling"
})
CREATE (e)-[:HAS_SEGMENT {order: 1}]->(s)
```

Entity layer — Person, Organization, Topic, and Event nodes capture the domain knowledge. Topics use a flat hierarchy with a `category` property (monetary_policy, war, religion, revolution) rather than deep taxonomic trees. Events have explicit dates and types (important battle, shift in power, election, crisis).

Analytical layer — Claim and Prediction nodes are **reified from relationships into first-class entities**, following Neo4j's official intermediate node pattern. A Claim carries `claimType` (factual, opinion, prediction, recommendation), `sentiment` (positive, negative, neutral), and `confidence` . A Prediction adds `targetDate` , `targetMetric` , `targetValue` , and crucially `outcome` (correct, incorrect, pending).

```
cypher

CREATE (c:Claim {
 claimId: "claim-245-001",
 text: "A Selic deve subir mais 50 pontos base na próxima reunião",
 claimType: "prediction", sentiment: "hawkish", confidence: 0.92
})
MATCH (guest:Person {name: "Marcos Lisboa"}), (u:Utterance {utteranceId: "utt-245-002"}) CREATE
(guest)-[:MADE_CLAIM]->(c)
CREATE (c)-[:STATED_IN]->(u)
CREATE (c)-[:ABOUT]->(:Topic {name: "Selic rate", category: "monetary_policy"})
```

Speaker attribution: the three-layer chain

The design enforces a **three-layer attribution chain**: `Person -[:SPOKE]-> Utterance <-[:HAS_UTTERANCE]- Segment <-[:HAS_SEGMENT]- Episode` . Claims link back through `[:STATED_IN]-> Utterance` , so you can always trace any claim to who said it, in which segment, of which episode, on what date. This is impossible with flat document-chunk architectures where speaker identity is lost in the embedding.

Modeling agreement, disagreement, and evolving narratives

Stance relationships connect claims, not people directly. When Guest B disagrees with Guest A's prediction, the graph records `(claimB)-[:DISAGREES_WITH {strength: 0.8}]- >(claimA)` . This enables tracking how the **same person's stance evolves** across episodes via `(claim_early)-[:EVOLVED_INTO]->(claim_later)` and `(prediction)-[:OUTCOME_DISCUSSED_IN]->(later_episode)` .

Temporal context is bi-temporal: every claim carries both "when said" (via Episode publishDate) and "what time period referenced" (via Event dates or Prediction targetDate). Timeline tree nodes (Year, Month) linked to Episodes enable efficient temporal traversal without expensive date property filtering.

Sizing and anti-patterns

For 300 episodes, expect approximately **140 Episode nodes, 2000 Segments, 60,000 Utterances, 200-500 Topics, 6,000-15,000 Claims, and 200,000-500,000 relationships**—comfortably handled by Neo4j Community Edition. Critical anti patterns to avoid:

never stuff rich text into relationship properties (use intermediate nodes), never create generic :RELATED_TO relationships (use specific types like :MADE_CLAIM , :ENDORSES , :DISPUTES for efficient traversal), and avoid super nodes by using topic hierarchies and temporal bucketing for high-connectivity topics.

2.5 Reification: claims and opinions as first-class graph structures

Modeling claims, opinions, and predictions as first-class nodes—rather than burying them as properties on edges—is the **reification pattern**, well-documented across both RDF and property graph traditions.

Neo4j's official Graph Data Modeling course documents the **intermediate node pattern**: replace a relationship that needs rich metadata with a node. For an employment relationship, instead of (Person)-[:WORKED_AT {from, to}]->(Company) , create an Employment node connected to both Person and Company via separate relationships.

```cypher
CREATE (claim:Claim {
 id: randomUUID(),
 text: 'Fed will cut rates by 50bp in March',
 type: 'prediction',
 confidence: 0.65,
 sentiment: 'dovish',
 createdAt: datetime('2025-02-15'),
 status: 'pending' // pending | verified | refuted
})
CREATE (speaker:Person {name: 'Janet Yellen'})-[:MADE_CLAIM {
 context: 'Senate testimony',
 timestamp: datetime()
}]->(claim)
CREATE (claim)-[:ABOUT {aspect: 'monetary policy'}]->(:Entity {name: 'Federal Reserve'}) CREATE
(claim2:Claim {text: 'Labor market too strong for rate cuts'})
CREATE (claim2)-[:CONTRADICTS {strength: 0.8}]->(claim)
CREATE (claim)-[:EXTRACTED_FROM]->(:Source {url: '...', type: 'transcript'})
```

Formal argumentation frameworks

The **Argument Interchange Format (AIF)** defines **I-Nodes** (information/claims), **S Nodes** (scheme applications subdivided into inference, conflict, and preference), and **L Nodes** (speech acts in dialogue). An AIF argument graph is a bipartite-like structure where edges always connect I-Nodes to S-Nodes and vice versa.

The **PAKT framework** (arXiv:2404.10570, 2024) extends AIF with Frames (perspectives), Values (underlying motivations), Stance, Camps (groups), and Zeitgeist (temporal context).

DialAM-2024 (Ruiz-Dolz et al., ArgMining Workshop at ACL 2024) presents the first shared task in dialogical argument mining using Inference Anchoring Theory. The framework models dialogue with locution nodes (what was said), proposition nodes (claims), and relation nodes (argumentative/illocutionary)—directly demonstrating how podcast speakers' claims and reasoning can be captured as first-class graph structures.

Joel Chan's Discourse Graphs (arXiv:2407.20666, 2024; further developed through Protocol Labs, validated at UIST 2024) model knowledge claims linked to supporting evidence through typed edges (supports, opposes, informs), with 3 years of practical deployment in distributed research communities.
In the RDF world, **RDF-star** (being standardized as RDF 1.2) allows quoted triples to serve as subjects of other triples, enabling lightweight statement-level provenance. The W3C RDF-DEV Community Group recommends using intermediate Claim nodes when multiple claims about the same triple need distinct metadata. **Fakih & Serrano Alvarado (Semantic Web Journal, 2024)** provide a comprehensive survey of all five reification models: standard reification, named graphs, n-ary relations, singleton properties, and RDF-star, comparing expressiveness, query complexity, and performance trade-offs. The **Ontotext blog (2024)** offers a practical conclusion: RDF star wins on performance when inference isn't critical; named graphs excel for inference use cases.

The **W3C PROV-O ontology** provides the definitive standard for provenance with three core classes: `Entity` (things), `Activity` (processes), and `Agent` (responsible parties), connected via `wasGeneratedBy` , `wasAttributedTo` , and `wasDerivedFrom` properties. Its **qualification pattern** adds intermediate classes (`Usage` , `Generation` , `Derivation`) for detailed provenance metadata.

Kialo (18,000+ debates, 720,000+ claims) uses a tree structure where each claim is either Pro or Con relative to its parent. **Argdown** provides a Markdown-like syntax for argument maps. Both demonstrate that argumentation graphs scale well in practice when claims are first-class nodes.

2.6 Temporal modeling: separating "when it was said" from "what it refers to"

Every system that models evolving knowledge from conversations must handle the fundamental distinction between **transaction time** (when a fact was recorded) and **valid time** (when a fact is true in the world). Graphiti's four-timestamp model (Section 2.2) is the most complete implementation in conversation graphs.

Academic foundations for bi-temporal graphs

Bitemporal Property Graphs (Anselma et al., ADBIS 2025) formally extends the AeonG model to support both valid time and transaction time with limited overhead. **AeonG** itself (Hou et al., PVLDB 2024; extended in VLDB Journal 2025) proposes temporal property graphs with hybrid storage achieving **5.73× lower storage and 2.57× faster temporal query latency**. **BiTRDF** (MDPI Mathematics, June 2025) adds valid/transaction time to standard RDF.

PROV-STAR (Dibowski, FOIS 2024) extends W3C PROV-O with RDF-star for triple-level change tracking, enabling full history and point-in-time recovery. Kleinsteuber et al. (Datenbank-Spektrum, 2024) compare storing provenance in the KG itself versus a relational database.

TG-RAG (arXiv:2510.13590, 2024-2025) implements a bi-level temporal graph where an upper layer of time nodes (quarters, years) connects to a lower layer of entity relation subgraphs. The same factual relation (e.g., "Western Digital → Revenue") exists as multiple time-stamped edges under different time nodes, preventing temporal confusion during retrieval.
For **temporal retrieval**, **STAR-RAG** (arXiv:2510.16715) constructs rule graphs summarizing event categories and temporal relations with personalized PageRank for time-aligned subgraph retrieval.

Prediction tracking schema

Extending Graphiti's model yields a natural schema for predictions: a PREDICTS edge from a Guest entity to an Outcome entity carries prediction_text , valid_at (when predicted), target_date (expected outcome date), confidence , and source_episode_id . A separate RESOLVED_AS edge later connects the Outcome to a Result with actual_value , resolved_at , and resolution_episode_id .

Neo4j offers built-in temporal types (DATE, DATETIME, DURATION) and documented patterns including **Timeline Trees** (Year→Month→Day→Event hierarchies) and **Entity State models** (entity linked to temporal State nodes via VALID_FROM/VALID_TO).

Speaker attribution via knowledge graph of character roles

A novel approach from **Expert Systems with Applications (2025)** uses a knowledge graph of character roles to improve speaker attribution to **91.9% exact match**, applicable to podcast transcript processing.

2.7 Discourse structure graphs

Beyond simple sequential chaining, academic research provides rich graph formalisms for modeling rhetorical and discourse relationships between utterances.

SDRT (Segmented Discourse Representation Theory), applied in the **STAC corpus** for multiparty dialogue, defines 17 discourse relations including Question-Elaboration, Explanation, Narration, Contrast, Acknowledgement, and Correction. Each Elementary Discourse Unit (EDU) becomes a graph node, with directed edges labeled by discourse relation type. Crucially, SDRT produces **graphs, not trees**: a single EDU can participate in multiple discourse relations simultaneously, essential for multi-party dialogue.

eRST (Computational Linguistics, 2025) unifies insights from RST, SDRT, PDTB, and CCR into a single formalism that allows tree-breaking relations while retaining nuclearity from RST—the state of the art for discourse graph annotation.

For **topic segmentation**, TopicSegment nodes group consecutive utterances sharing a topic. Xu et al. (2024) demonstrated mutual learning between rhetorical structure parsing and topic segmentation using Graph Attention Networks. When a conversation returns to a previous topic:

- **Graphiti communities**: Entity mentions across episodes resolve to the same Entity nodes; community membership persists, so new episodes link to existing entities
- **Di Bratto ring patterns**: Distant utterances referencing the same entity create cycles in the graph, capturing topic recurrence
- **DialogueGCN self-dependency edges**: Long-range edges between same-speaker utterances capture thematic threads across turns

For **cross-turn assertions**, Graphiti's approach assembles each entity edge's fact field from information across one or more episodes, with the episodes list tracking source episodes. When later information refines or contradicts an earlier fact, the old edge is invalidated and the fact text regenerated.

2.8 Real-world provenance schemas

Several production systems demonstrate graph schemas for tracking attributed statements

with confidence and temporal metadata.

GDELT's Global Knowledge Graph processes news across 65 languages, updating every 15 minutes. Its three-table architecture maps naturally to a graph: Event nodes connect to Actor entities via involvement relationships, while a Mentions table records each citation with **confidence scores and character offsets**. The Quotations field extracts attributable quotes. Researchers (arXiv:2503.07584, 2025) built a formal ontology mapping GDELT to a knowledge graph.

Palantir Gotham uses a three-layer ontology: a semantic layer (Object Types with typed Link Types), a kinetic layer (actions/workflows), and a dynamic layer (AI inference). Every data point maintains provenance to its original source, with access restrictions at the individual attribute level, captured in **tamper-evident audit logs**.

In the **legal domain**, NyOn (ESWC 2022) models Indian court judgments with entity types Case , Judge , Petitioner , Respondent , Advocate , Statute , and Section . A Legal RAG system (arXiv:2502.20364, February 2025) combines Neo4j with Milvus for hybrid retrieval.

Medical consultation KGs demonstrate per-turn graph evolution. The MedKP framework (arXiv:2403.06611, 2024) retrieves relevant subgraphs based on entities per dialogue turn, tracking physician actions as a clinical pathway. The RECAP-KG system mines knowledge graphs from physician notes using SNOMED-CT ontology mapping.

For **financial prediction tracking**, the stock_skills project (GitHub) implements a Neo4j schema with Stock , Report , Research , Note , and Trade nodes connected by AI generated relationships (INFLUENCES , CONTRADICTS , SUPPORTS , SUPERSEDES) carrying confidence scores and semantic embeddings.

2.9 No unified dialogue ontology exists, but the pieces are available

The most significant gap is the absence of a comprehensive formal ontology combining discourse structure, dialogue acts, entity extraction, temporal metadata, and conversation hierarchy. The closest existing ontologies serve complementary purposes:

- **SIOC** (W3C Member Submission): Most mature formal OWL/RDF ontology for threaded conversations, modeling Site , Forum , Post , and UserAccount . Covers metadata but not discourse relations or dialogue acts
- **Schema.org Conversation**: Conversation contains Message instances with sender , recipient , about , datePublished . Too flat—no discourse structure • **SDRT/AIF**: Formal discourse relations and argumentation structure, but not designed for database-level modeling
- **PROV-O**: Definitive provenance tracking but not dialogue-specific

The **ISWC 2025 OKE-D workshop** represents the most recent academic effort toward dialogue-aligned knowledge graphs with formal RDF alignment and Wikidata entity linking. **SGMem** (arXiv:2509.21212, 2025) introduces sentence-level graph memory for long-term conversational agents.

Synthesized schema combining best patterns

```
(:Corpus)-[:CONTAINS]->(:Conversation)
(:Conversation)-[:HAS_EPISODE]->(:Episode)
(:Episode)-[:NEXT_EPISODE]->(:Episode) // temporal chain
(:Episode)-[:HAS_SEGMENT]->(:TopicSegment)
(:TopicSegment)-[:CONTAINS]->(:Utterance)
(:Utterance)-[:NEXT]->(:Utterance) // turn chain
(:Utterance)-[:SPOKEN_BY]->(:Speaker) // attribution
(:Utterance)-[:MENTIONS]->(:Entity) // entity grounding
(:Utterance)-[:EXPRESSES]->(:Claim) // reified claims
(:Claim)-[:SUPPORTS|CONTRADICTS]->(:Claim) // argumentation
(:Claim)-[:ABOUT]->(:Entity) // claim targets
(:Entity)-[r:RELATION]->(:Entity) // semantic facts
 // r carries: fact, embedding, valid_at, invalid_at, created_at, expired_at
(:Entity)-[:IN_COMMUNITY]->(:Community) // topic clusters
```

This combines Graphiti's temporal architecture, Di Bratto's utterance chaining, the lexical graph's dual-path retrieval, AIF's argumentation structure, and PROV-O's provenance tracking.

2.10 Other conversation KG systems

Feature	neo4j-graphrag	Graphiti/Zep	Cognee	Mem0
Primary focus	GraphRAG pipeline + retrievers	Temporal conversational KG	AI memory engine (ECL)	Persistent memory layer
Graph backends	Neo4j only	Neo4j, FalkorDB, KuzuDB	Neo4j (optional)	
Temporal awareness	No	Yes (core feature)	Limited	No
Conversation memory	Not specialized	Core use case	Supported Core use case	Core use case (Neo4j-backed)
Incremental updates	Manual pipeline	Dynamic graph updates	KG rebuilds	Append to stores
Maturity	Production	20K+ GitHub stars	7K+ GitHub stars	YC-backed

Additional systems: **Cognee** claims outperformance on HotPotQA but benchmarks were self-conducted; its graph analysis reveals Mem0 produces disconnected entity clusters missing cross-sentence links, while Graphiti uses generic relations. **Read AI** (proprietary, 5M+ monthly active users) provides cross-channel knowledge graph across meetings, emails, and messages. **MemGPT/Letta** uses an OS-inspired memory hierarchy rather than a native knowledge graph—users have requested native KG support but it remains unimplemented.

3. The Multi-Pass Extraction Pipeline

Single-pass extraction—sending full text to an LLM and asking it to extract everything at once—is documented to miss entities and produce suboptimal results. The research converges on multi-pass extraction as strictly superior.

3.1 Benchmarks and quantitative evidence for extraction quality

Text2KGBench (Mihindukulasooriya et al., ISWC 2023) remains the foundational benchmark for ontology-driven KG generation, but **Text2KGBench-LettrIA** (CEUR-WS Vol-4041, LLM-TEXT2KG 2025 Workshop) substantially revised it with 14,000+ re annotated triples and refined ontologies, demonstrating that schema refinement directly impacts extraction quality evaluation.

Multi-pass extraction delivers substantial improvements

Multiple 2024–2025 papers quantify the improvement:

- **ECR (Extract-Critique-Refine)**: **+16% improvement in triple F1** over single pass
- **CORE-KG ablation**: Removing coreference resolution causes a **28.32% increase in node duplication**; removing structured prompts causes **73% more noise • SAC-KG** (Chen et al., ACL 2024): Three-component Generator→Verifier→Pruner pipeline achieves **89.32% precision** at 1M+ node scale, a **>20% increase over SOTA**
- **KGGen** (Mo et al., NeurIPS 2025): Multi-stage extract→cluster→normalize scores **66.07% on MINE** versus GraphRAG's 47.80% and OpenIE's 29.84%—an **18% improvement**. Claude Sonnet 3.5 achieved the highest individual score of 73% • **Graphusion** (Yang et al., arXiv:2410.17600, 2024): Three-step seed→candidate→fusion achieves **9.2% accuracy improvement** on sub-graph completion and **2.92/3 human-evaluated entity extraction quality • iText2KG** (Lairgi et al., WISE 2024): Four-module pipeline with separated entity and relation extraction. The ATOM extension achieves **~31% gain in factual exhaustivity, ~18% temporal exhaustivity, and 93.8% latency reduction • UrbanKGent** (NeurIPS 2024): Iterative trajectory refinement **surpasses GPT-4 by >10%** at approximately **20× lower cost**

3.2 Pass architecture and model tiering

The pipeline uses five sequential passes with tiered model selection: **Pass**

Task	Model	Why
1 Entity extraction	GPT-5-mini	High volume, structured output, cheap
2 Relationship extraction	GPT-5-mini	Same chunks, entities as context
3 Claims/opinions	GPT-5 or Claude	Needs nuanced stance/sentiment understanding
4 Coreference resolution	GPT-5 or Claude	Deep contextual reasoning required
5 Entity resolution	Embeddings + GPT-5- mini	Only ~5% of pairs need LLM judgment

3.3 Structured outputs guarantee schema compliance

OpenAI's Structured Outputs (August 2024) use constrained decoding to guarantee **100% JSON schema adherence** versus <40% for GPT-4-0613. Define Pydantic models for extraction outputs:

python

```
from pydantic import BaseModel, Field
from enum import Enum

class EntityType(str, Enum):
 PERSON = "person"
 ORGANIZATION = "organization"
 TOPIC = "topic"
 EVENT = "event"
 CLAIM = "claim"

class Entity(BaseModel):
 name: str = Field(description="Canonical entity name")
entity_type: EntityType
 description: str = Field(description="Brief description from context")
speaker_attribution: str | None = Field(default=None)  aliases: list[str] =
Field(default_factory=list)

class Relationship(BaseModel):
 source_entity: str
 target_entity: str
 relationship_type: str
 description: str
 confidence: float = Field(ge=0.0, le=1.0)
 evidence: str

class ExtractionResult(BaseModel):
 entities: list[Entity]
 relationships: list[Relationship]
```

Call via OpenRouter with strict schema enforcement:

python

```
response = httpx.post(
    "https://openrouter.ai/api/v1/chat/completions",
 headers={"Authorization": f"Bearer {OPENROUTER_KEY}"},
 json={
        "model": "openai/gpt-5-mini",
        "messages": [
            {"role": "system", "content": EXTRACTION_PROMPT},
            {"role": "user", "content": chunk_text},
```

```
        ],
        "response_format": {
            "type": "json_schema",
            "json_schema": {
                "name": "ExtractionResult",
                "strict": True,
                "schema": ExtractionResult.model_json_schema(),
            }
        },
        "temperature": 0.0,
    },
)
```

Constrained decoding benchmarks

JSONSchemaBench (Geng et al., arXiv:2501.10868, ICML 2025 Workshop) benchmarks six frameworks across 10K real-world schemas: constrained decoding generally improves output quality by up to **4% on downstream tasks**, not just format compliance. **XGrammar** (Dong et al., MLSys 2025) achieves token mask generation in **<40 microseconds per token** and is the default backend for vLLM. **llguidance** (Microsoft) achieves ~50µs CPU time per token—OpenAI publicly credited it for foundational work underlying their Structured Outputs. The **Instructor library** (11K+ GitHub stars, 3M+ monthly PyPI downloads) provides production-ready Pydantic-based structured extraction with automatic validation and retries across 15+ LLM providers.

3.4 Extraction prompts with speaker context

The entity extraction prompt must include Episode context and domain-specific entity types.

```
python
You are an expert historian and geopolitics analyst extracting entities from a YouTube history class transcript ("Predictive History" by Prof. Jiang).

ENTITY TYPES: person, organization, topic, event, claim
EPISODE CONTEXT: {known entities/subjects/focus for this episode}

RULES:
- Use universally recognized canonical names (e.g., "Alexander the Great", "Roman Empire").
- Include ALL aliases, titles, and abbreviations.
- Attribute every entity and claim to the speaker who mentioned it.
- For PERSON, classify role: (ruler / military_leader / philosopher / diplomat / civilian).
- For ORGANIZATION, treat historical polities as organizations. Classify type: (empire / nation-state / dynasty / alliance / religion / institution).
- For TOPIC, assign category: (geography / ideology / economic_system / governing_system / military_strategy / religious_practice).
- For EVENT, classify type: (war / battle / treaty / revolution / crisis / period).
- For CLAIM, extract specific historical interpretations, geopolitical predictions, or analytical assertions made by the speaker (e.g., "The collapse of the Bronze Age was driven by supply chain failures").

TRANSCRIPT CHUNK:
{chunk_text}
```

The claims extraction pass uses a separate, more nuanced prompt:

```
python

Extract notable claims, geopolitical predictions, historical interpretations, and analytical assertions from this history class transcript.

For each claim identify:

- claim_text: The claim as a clear standalone statement

- claim_type: factual_assertion | historical_interpretation | geopolitical_prediction | causal_analysis

- stance: critical | favorable | neutral | speculative

- entities_involved: List of entities referenced (must map to your extracted person, organization, topic, or event entities)

- time_reference: What specific historical era, date, or future timeline the claim references (if any)

Focus on: causal explanations of historical events, geopolitical predictions, analysis of power dynamics, interpretations of leaders' motives, and theories on historical cycles.

TRANSCRIPT: {text}
```

3.5 Segment-aware chunking improves semantic extraction

Standard fixed-size chunking destroys segment boundaries. We do not want to cut a chunk when the orator is in the middle of conveying an idea. The chunking strategy must employ a two step chunking logic. One step to best try and determine where an explanation starts and ends so that we can appropriately extract semantic value for the next step of chunking where we in fact chunk the information for proper extraction. So LEVEL 1 CHUNK refers to the first pass LEVEL 2 CHUNK refers to the proper 1500 token max chunk.

Below is a function used for some inspiration on what we want to achieve. The function below is part of a project where we use a chunking strategy that groups complete speaker turns into chunks of ~1,500 tokens with 2-turn overlap, ensuring every chunk preserves who is speaking. Since we are not dealing with speaker turns in the Jiang project, its just food for

thought.

BELOW IS AN EXAMPLE OF CHUNKING IN A SPEAKER ATTRIBUTION CONTEXT, FOR OUR USE CASE WE WILL PROBABLY WANT TWO FUNCTIONS FOR A TWO PASS CHUNKING EFFORT.

Naive chunking fails for multi-speaker content. Cohere's chunking guide (2025) explicitly demonstrates that naïve fixed-size chunking can separate a speaker's name from their statement, causing RAG failures. Microsoft Azure's context-aware RAG system (2025) produces JSON chunks with sectiontitle , speaker , and content fields, claiming **80–85% reduction in token usage**. The MLPills tutorial (2024) provides a hands-on pipeline for building podcast-to-knowledge-graph systems on Neo4j.

Speaker-turn-aware chunking groups complete speaker turns into chunks of ~1,500 tokens with 2-turn overlap:

```
python

def chunk_by_speaker_turns(transcript: str, max_tokens: int = 1500,
 overlap_turns: int = 2) -> list[dict]:
 turns = parse_speaker_turns(transcript) # Split on "Speaker:" patterns
 chunks, i = [], 0
    while i < len(turns):
 chunk_turns, token_count, j = [], 0, i
        while j < len(turns) and token_count < max_tokens:
 turn_tokens = estimate_tokens(turns[j]["text"])
            if token_count + turn_tokens > max_tokens and chunk_turns:
                break
 chunk_turns.append(turns[j])
 token_count += turn_tokens
 j += 1
 chunks.append({
            "text": "\n\n".join(t["text"] for t in chunk_turns),
            "speakers": list(set(t["speaker"] for t in chunk_turns)),
            "start_turn": i, "end_turn": j - 1
        })
 i = max(j - overlap_turns, i + 1)
    return chunks
```

3.7 Entity resolution cascades from cheap to expensive

Entity deduplication uses a four-stage cascade that routes only the hardest cases to the LLM:

1. **Exact match** — normalize case, whitespace, accents (covers ~40% of duplicates) 2. **Fuzzy string matching** — token_sort_ratio > 85 on same-type entities using rapidfuzz (covers ~35%)
3. **Embedding similarity** — cosine > 0.85 between entity name+description embeddings via FAISS semantic blocking (covers ~20%)

4. **LLM judgment** — only ambiguous cases (~5% of pairs) go to GPT-5-mini for final merge decisions

LINK-KG's research demonstrated this cascade reduces node duplication by **45%** compared to no resolution. Graph clustering via Louvain prevents over-merging from transitive closure. A practical cascade architecture detailed by **Shereshevsky (Towards Data Science, January 2026)**: Tier 1 fast rules handle **40% of decisions** (exact ID match), Tier 2 ML models handle **55%** (Ditto with confidence thresholds), Tier 3 LLMs handle **5%** (ambiguous pairs).

Peeters & Bizer (EDBT 2025) demonstrate GPT-4 **outperforms fine-tuned PLMs by 40–68% on unseen entity types** in zero-shot entity matching. **"The Rise of Semantic Entity Resolution"** (Jurney, Towards Data Science, 2025) argues entity resolution for KGs is fundamentally link prediction of SAME_AS edges, emphasizing embedding-based blocking followed by LLM matching. **COLING 2025** investigates match, compare, and select strategies. The Connected Data London 2024 ERKG Masterclass (Paco Nathan, DerwenAI) demonstrates failure modes when GraphRAG operates without proper entity resolution.

The **neo4j-graphrag Python package** supports production entity resolution with built in SpaCy (semantic matching) and RapidFuzz (fuzzy matching).

3.8 Cost optimization makes 300 episodes affordable

Three mechanisms stack to reduce costs dramatically (up to **~75% cost reduction**):

Prompt caching (50% discount on repeated prompt prefixes > 1,024 tokens) works because the system prompt and schema are identical across all chunks—place static instructions first, variable chunk text last. LightRAG Issue #2355 details restructuring extraction prompts for ~45% cost reduction; the Thomson Reuters blog notes sequential processing with shared cache prefixes dramatically outperforms parallel in cost efficiency.

Batch API (50% discount, 24-hour completion) is ideal for processing the historical backlog.

Tiered model usage keeps the cheap model handling 80% of extraction volume. SAP's CIKM 2025 paper demonstrates a dependency-parsing pipeline achieving **94% of LLM based performance at dramatically lower cost** by eliminating LLM calls for entity extraction entirely.
Estimated cost per 2-hour episode (~33K tokens): approximately **$0.05-0.15 with GPT-5-mini** as the primary extraction model. For 300 episodes: **$15-45 total**—orders of magnitude cheaper than Microsoft GraphRAG's documented $33K.

4. The Hybrid Retrieval Pipeline

Retrieval must combine the semantic power of vector search with the structural precision of graph traversal.

4.1 Neo4j's retrieval stack

The **neo4j-graphrag Python package** (v1.13.0) provides four key retrievers: `VectorRetriever` , `VectorCypherRetriever` , `HybridRetriever` , and `HybridCypherRetriever` . The **HybridCypherRetriever** is

most relevant—it adds a Cypher graph traversal step after hybrid (vector + full-text) search. Alex Thomas's Neo4j Developer Blog series (2024) demonstrates how vector search alone misses domain-specific terms that keyword search captures. The Text2CypherRetriever handles natural language to Cypher translation. LangChain's Neo4j integration supports search_type="hybrid" with custom retrieval_query .

4.2 Vector indexes and graph traversal fan-out

Neo4j 5.x+ supports native vector indexes:

cypher

```
CREATE VECTOR INDEX `chunk-embeddings` IF NOT EXISTS
FOR (c:Chunk) ON (c.embedding)
OPTIONS {indexConfig: {
  `vector.dimensions`: 1536,
  `vector.similarity_function`: 'cosine'
}};

CREATE VECTOR INDEX `entity-embeddings` IF NOT EXISTS
FOR (e:Entity) ON (e.embedding)
OPTIONS {indexConfig: {
  `vector.dimensions`: 1536,
  `vector.similarity_function`: 'cosine'
}};
```

The core retrieval pattern uses **vector search as the entry point, then Cypher traversal to fan out**:

python

```
def hybrid_retrieve(query: str, top_k: int = 5):
 query_embedding = get_embedding(query)
 results = session.run("""
 // Stage 1: Vector search finds entry-point chunks
 CALL db.index.vector.queryNodes('chunk-embeddings', $top_k, $embedding)  YIELD node
AS chunk, score

 // Stage 2: Fan out into graph neighborhood
 OPTIONAL MATCH (chunk)<-[:HAS_UTTERANCE]-(seg:Segment)<-[:HAS_SEGMENT]-(ep:Episod OPTIONAL
MATCH (p:Person)-[:SPOKE]->(chunk)
 OPTIONAL MATCH (claim:Claim)-[:STATED_IN]->(chunk)
 OPTIONAL MATCH (claim)-[:ABOUT]->(topic:Topic)

 // Stage 3: Assemble structured context
 RETURN chunk.text AS text, score, ep.title AS episode, ep.publishDate AS date,  p.name AS
speaker,
 collect(DISTINCT {claim: claim.text, type: claim.claimType,
 sentiment: claim.sentiment, topic: topic.name}) AS claims  ORDER BY score DESC
 """, embedding=query_embedding, top_k=top_k)
```

```
    return [dict(r) for r in results]
```

Neo4j's VectorCypherRetriever formalizes this pattern:

```
cypher

-- Hybrid retrieval: vector finds chunks, graph expands context
WITH node AS chunk
MATCH (chunk)<-[:FROM_CHUNK]-(entity)-[relList:!FROM_CHUNK]-{1,2}(nb) UNWIND
relList AS rel
WITH collect(DISTINCT chunk) AS chunks, collect(DISTINCT rel) AS rels
RETURN apoc.text.join([c in chunks | c.text], '\n') + '\n' +
 apoc.text.join([r in rels | type(r)+': '+startNode(r).name+' -> '+endNode(r).name], '\n') AS output
```

4.3 Query-type routing with specialized strategies

A lightweight classifier (GPT-5-mini, zero temperature) categorizes incoming queries and routes to the optimal retrieval strategy.
Adaptive-RAG (Jeong et al., NAACL 2024) is the seminal paper on query-adaptive retrieval, training a classifier to predict query complexity: no retrieval needed, single step RAG, or multi-step iterative RAG. **RAGRouter** (Zhang et al., arXiv:2505.23052, May 2025) models how retrieved documents shift each LLM's knowledge, outperforming individual LLMs by **1.67–9.33%**. **MBA-RAG** (Tang et al., arXiv:2412.01572, December 2024) reframes strategy selection as a multi-armed bandit problem. Enterprise deployments report **35% P50 latency reduction** and **28% LLM API cost reduction** from adaptive routing.

Factual queries ("What did Julius Caesar did in Egypt?") use entity lookup with neighborhood traversal:

```
cypher

CALL db.index.vector.queryNodes('chunk-embeddings', 20, $queryEmbedding) YIELD
node AS chunk, score
MATCH (chunk)<-[:HAS_UTTERANCE]-(seg)<-[:HAS_SEGMENT]-(ep)
MATCH (p:Person {name: $figureName})-[:SPOKE]->(chunk)
RETURN chunk.text, score, ep.title, ep.publishDate
ORDER BY score DESC LIMIT 5
```

Comparative queries ("How do persons A and B differ on fiscal policy?") retrieve claims from both speakers side by side:

```
cypher

MATCH (p:Person)-[:MADE_CLAIM]->(c:Claim)-[:ABOUT]->(t:Topic)
WHERE p.name IN [$guestA, $guestB] AND toLower(t.name) CONTAINS toLower($topic) MATCH
(c)-[:STATED_DURING]->(ep:Episode)
RETURN p.name AS guest, c.text AS claim, c.sentiment, ep.publishDate
ORDER BY p.name, ep.publishDate
```

Temporal queries ("How has the view on inflation evolved?") aggregate claims chronologically:

```cypher
MATCH (c:Claim)-[:ABOUT]->(t:Topic {name: $topic})
MATCH (c)-[:STATED_DURING]->(ep:Episode)
MATCH (p:Person)-[:MADE_CLAIM]->(c)
RETURN ep.publishDate AS date, p.name AS speaker, c.text AS claim, c.sentiment ORDER
BY date ASC
```

Thematic queries ("What are the main investment themes across all episodes?") use community summaries.

4.4 Text-to-Cypher with self-healing retry

For structured queries that map naturally to graph patterns, the system generates Cypher from natural language with the full graph schema, few-shot examples, and strict read-only constraints. When a generated query fails, the error message and schema are fed back to the LLM—up to 2 retries.

The **Neo4j Text2Cypher 2024 dataset** (Ozsoy et al.) comprises **44,387 instances** (39,554 train / 4,833 test) combining 16 public datasets. **GPT-4o achieved best performance**; fine-tuned GPT-4o achieved **>30% ExactMatch** in execution-based evaluation. Text2Cypher is described as "the most flexible, but also the most unreliable" retrieval pattern. A July 2025 paper (MDPI Applied Sciences) proposes **GRPO-based reinforcement learning** to improve Text2Cypher on smaller models.

4.5 Reciprocal Rank Fusion for result assembly

The original **RRF paper** (Cormack et al., SIGIR 2009) with formula $RRF_score(d) = \Sigma\ 1/(k + rank(d))$ and k=60 remains the standard. **RAG-Fusion** (Rackauckas, arXiv:2402.03367, 2024) extends RRF to RAG by generating multiple query variations, reporting **+8–10% accuracy and +30–40% comprehensiveness**. A PostgreSQL implementation saw pure vector search at **~62% precision jump to ~84%** with RRF fused hybrid search.

Recommended architecture: Stage 1 uses hybrid search fused with RRF for high recall (top 100), Stage 2 uses cross-encoder reranking for precision (top 5–10).

4.6 Context assembly and generation

Retrieved results from all active strategies are fused and separated into raw text passages and structured graph context:

```python
def assemble_context(vector_results, graph_context, t2c_results=None):
 parts = ["## Relevant Passages"]
    for r in vector_results[:5]:
 parts.append(f"[{r['speaker']}, {r['episode']}, {r['date']}]: {r['text']}")
```

```
    parts.append("\n## Knowledge Graph Context")
    for r in graph_context:
        for claim in r.get('claims', []):
            parts.append(f" - {claim['topic']}: {claim['claim']} ({claim['sentiment']})")
    if t2c_results and t2c_results.get('results'):
        parts.append("\n## Structured Query Results")
        for row in t2c_results['results'][:10]:
            parts.append(str(row))

    return "\n".join(parts)
```

5. Community Detection and Thematic Understanding

5.1 Leiden community detection on Neo4j GDS

Microsoft's foundational **GraphRAG paper** (Edge et al., arXiv:2404.16130, April 2024) establishes hierarchical Leiden community detection as the standard for partitioning entity knowledge graphs into thematic communities.

The workflow requires three steps: project the in-memory graph, run Weakly Connected Components to assess connectivity, then run Leiden with hierarchical output:

```cypher
-- Project entity co-occurrence graph
CALL gds.graph.project(
  'podcast-kg',
  ['Entity', 'Topic', 'Person'],
  {RELATED_TO: {orientation: 'UNDIRECTED', properties: ['weight']}}
)

-- Run Leiden with hierarchy
CALL gds.leiden.write('podcast-kg', {
 writeProperty: 'communityId',
 includeIntermediateCommunities: true,
 relationshipWeightProperty: 'weight',
 randomSeed: 42
})
YIELD communityCount, modularity, ranLevels
```

The `includeIntermediateCommunities: true` parameter returns communities at **all hierarchy levels**, enabling both fine-grained topic clusters (Level 0) and broad thematic areas (higher levels). The `gamma` parameter (default 1.0) controls resolution.

5.2 GPU acceleration and incremental methods

NVIDIA's **cuGraph Leiden implementation** achieves **47.5× speedup** over graspologic and **8.8×** over igraph on graphs with 3.8M nodes/16.5M edges.

For incremental updates, **Dynamic Frontier Leiden** (Sahu, arXiv:2405.11658, May 2024) processes only affected vertices after batch updates, achieving comparable modularity to static

Leiden at dramatically reduced runtime—the most directly applicable algorithm for incremental community detection in GraphRAG. **DF Louvain** (Sahu, arXiv:2404.19634) achieves **179× speedup** over static Louvain. For real-time scenarios, Wang et al. (KGSWC 2025) achieve sub-second to sub-minute detection on billion-scale graphs.

5.3 Community summarization with LLMs

For each detected community, gather members and generate a thematic summary:

```python
def summarize_community(community_id, level):
 members = session.run("""
 MATCH (e:Entity) WHERE e.communities[$level] = $cid
 OPTIONAL MATCH (e)-[r]->(other:Entity)
 WHERE other.communities[$level] = $cid
 RETURN e.name, e.description, type(r), r.description, other.name
 """, cid=community_id, level=level)

 context = format_community_members(members)
 summary = llm.invoke(f"""
 Summarize this community of related entities from a History class.  Include: key themes,
important claims, and significance. {context}""")

    # Store with embedding for vector retrieval
 session.run("""
 MERGE (c:Community {id: $cid, level: $level})
 SET c.summary = $summary, c.embedding = $embedding
 """, cid=community_id, level=level, summary=summary,
 embedding=get_embedding(summary))
```

5.4 Incremental updates when new episodes arrive

Neo4j GDS's seedProperty enables **seeded Leiden**:

```cypher
CALL gds.leiden.write('podcast-kg-updated', {
 writeProperty: 'communityId',
 seedProperty: 'communityId', -- reuse existing assignments
 includeIntermediateCommunities: true
})
```

Only communities whose membership changed need re-summarization. For a single new episode added weekly, this means re-summarizing **5-15 communities** rather than all of them.

6. End-to-End Architecture and Production Patterns

6.1 The complete pipeline flow

Transcript → Clean PT-BR text → Speaker-aware chunking (1500 tokens, 2-turn overlap) → Pass 1: Entity extraction (GPT-5-mini, structured output)
→ Pass 2: Relationship extraction (GPT-5-mini, entities in context)
→ Pass 3: Claim/opinion extraction (GPT-5/Claude, nuanced stance detection) → Entity resolution (exact → fuzzy → embedding → LLM cascade)
→ MERGE into Neo4j (idempotent upsert with ON CREATE / ON MATCH) → Generate embeddings (OpenAI text-embedding-3-small)
→ Detect communities (Leiden via GDS)
→ Summarize communities (LLM)
→ Ready for retrieval

6.2 Project structure

```
podcast-graphrag/
├── src/
│   ├── config.py # Neo4j, OpenRouter, embedding settings
│   ├── preprocessing/
│   │   ├── text_cleaner.py # PT-BR normalization, filler removal
│   │   └── chunker.py # Speaker-aware chunking
│   ├── extraction/
│   │   ├── prompts.py # All extraction prompts
│   │   ├── schemas.py # Pydantic models for structured output
│   │   ├── extractor.py # Multi-pass extraction orchestration
│   │   └── resolver.py # Entity resolution cascade
│   ├── graph/
│   │   ├── neo4j_client.py # Connection, constraints, indexes
│   │   ├── upsert.py # MERGE patterns
│   │   └── communities.py # Leiden + summarization
│   ├── retrieval/
│   │   ├── classifier.py # Query type classification
│   │   ├── vector_search.py # Vector + graph fan-out
│   │   ├── text2cypher.py # Natural language → Cypher
│   │   └── assembler.py # Context assembly + re-ranking
│   ├── generation/
│   │   └── generator.py # Final answer generation
│   └── mcp/
│       └── server.py # MCP server for Claude/client access
├── scripts/
│   ├── ingest_episode.py # Process single new episode
│   ├── full_rebuild.py # Initial backlog processing
│   └── validate_graph.py # Quality assurance checks
└── data/cache/ # LLM response cache (prompt_hash → result)
```

6.3 Idempotent graph upserts

The critical Neo4j rule: **never MERGE entire patterns with unbound variables.** Always bind nodes first, then MERGE relationships between bound nodes:

cypher

```
-- Correct: Entity upsert with description enrichment
UNWIND $entities AS entity
MERGE (e:Entity {id: entity.id})
ON CREATE SET e.name = entity.name, e.type = entity.type,
 e.description = entity.description, e.source_count = 1
ON MATCH SET e.description = CASE
    WHEN size(e.description) < size(entity.description)
    THEN entity.description ELSE e.description END,
 e.source_count = e.source_count + 1

-- Correct: Relationship upsert between bound nodes
UNWIND $relationships AS rel
MATCH (source:Entity {id: rel.source_id})
MATCH (target:Entity {id: rel.target_id})
MERGE (source)-[r:RELATED_TO {type: rel.type}]->(target)
ON CREATE SET r.description = rel.description, r.weight = 1
ON MATCH SET r.weight = r.weight + 1
```

6.4 Quality assurance and validation

Run structural validation queries after each ingestion:

```
cypher

-- Orphan entities (no relationships = potential noise)
MATCH (e:Entity) WHERE NOT (e)--() RETURN count(e) AS orphans;

-- Claims without speaker attribution
MATCH (c:Claim) WHERE NOT (c)<-[:MADE_CLAIM]-(:Person) RETURN count(c);

-- Duplicate relationships
MATCH (a)-[r1:RELATED_TO]->(b), (a)-[r2:RELATED_TO]->(b)
WHERE id(r1) < id(r2) AND r1.type = r2.type RETURN count(r1) AS duplicates;
```

Entities mentioned in only one chunk across the entire corpus are likely noise and should be flagged. A **confidence threshold of 0.7** on extraction results filters low quality triples.

KGGen's MINE framework (Stanford, NeurIPS 2025) provides two complementary metrics: MINE-1 measures knowledge retention, MINE-2 measures RAG effectiveness. **LLM-as-validator** approaches (ScienceDirect, 2025) find GPT-4o and Claude achieve expert-level ontology axiom validation. **KARMA** (NeurIPS 2025) uses multi-agent architecture where specialized agents cross-verify outputs through LLM-based debate. **Ghanem & Cruz (Frontiers in Big Data, 2025)** introduce metrics including graph match (F1 ≥ 0.95), graph edit distance, hallucination rates per triple, and completeness.
Particula Tech's case study (**12M nodes, 89M relationships**) recommends 3-hop default traversal depth (5-hop for analytical), 500-node result caps.

6.5 LLM response caching

Cache every LLM call by hash(prompt + params) to disk. This provides cost protection against re-runs, enables debugging by inspecting cached responses, and makes the pipeline **idempotent**—reprocessing an episode produces identical results without additional API costs.

6.6 Incremental updates avoid costly full rebuilds

Microsoft GraphRAG's incremental indexing caches atomic steps but requires recomputing community detection and summaries. A production case study (Particula Tech, 2025) went from **14-hour rebuilds to 8-minute incremental updates** using document-level version tracking and change detection.

IncRML (Van Assche et al., Semantic Web Journal, 2024) demonstrates incremental KG construction achieving **315× less storage and 4.41× faster generation** versus full rebuilds.

6.7 MCP server exposes the graph to client-side LLMs (A LOT OF THE FUNCTIONS BELOW ARE JUST EXAMPLES FROM A SPEAKER ATTRIBUTION ORIENTED RESEARCH, FOR OUR PURPOSES IT WILL BE DIFFERENT KINDS OF QUESTIONS)

Anthropic announced MCP in November 2024 as an open standard for connecting AI systems to external data. The **v2025-11-25 specification** defines three primitives (Tools, Resources, Prompts) over JSON-RPC 2.0 with OAuth-based authorization. MCP was donated to the **Agentic AI Foundation** under the Linux Foundation in December 2025, co-founded by Anthropic, Block, and OpenAI, with adoption by Google DeepMind and Microsoft. By late 2025, MCP had **97M monthly SDK downloads and 10,000+ servers**.

Neo4j provides three official MCP servers (neo4j-contrib/mcp-neo4j on GitHub): a Cypher server for schema introspection and read/write queries, a Memory server for entity/relationship storage with observation tracking, and an Aura provisioning server.

For custom access, expose key retrieval operations as MCP tools:

```python
from mcp import FastMCP

mcp = FastMCP("podcast-graphrag")

@mcp.tool()
async def search_podcast(query: str, top_k: int = 5) -> str:
    """Semantic search across podcast knowledge graph."""
    results = hybrid_retrieve(query, top_k)
    return format_results(results)

@mcp.tool()
async def find_speaker_claims(speaker: str, topic: str = None) -> str:
    """Find all claims made by a specific speaker, optionally filtered by topic."""
    cypher = """
    MATCH (p:Person)-[:MADE_CLAIM]->(c:Claim)
    WHERE toLower(p.name) CONTAINS toLower($speaker)
    OPTIONAL MATCH (c)-[:ABOUT]->(t:Topic)
    RETURN p.name, c.text, c.claimType, c.sentiment, t.name AS topic
    """
```

```
    results = session.run(cypher, speaker=speaker)
    return format_results(results)

@mcp.tool()
async def compare_views(topic: str) -> str:
    """Compare different speakers' views on a topic across episodes."""
    results = session.run("""
    MATCH (p:Person)-[:MADE_CLAIM]->(c:Claim)-[:ABOUT]->(t:Topic)
    WHERE toLower(t.name) CONTAINS toLower($topic)
    MATCH (c)-[:STATED_DURING]->(ep:Episode)
    RETURN p.name, c.text, c.sentiment, ep.publishDate
    ORDER BY ep.publishDate
    """, topic=topic)
    return format_results(results)
```

6.8 Additional meeting and dialogue research

FAME (Kirstein et al., Findings of ACL 2025) generates 500 English + 300 German synthetic meetings with psychologically grounded participants. **Re-FRAME** (Findings of EMNLP 2025) proposes fact-based meeting summarization reducing hallucination by 2/5 points on the MESA metric. **Chatty-Gen** (Omar et al., arXiv:2501.09928, 2025) demonstrates the bidirectional KG↔dialogue pipeline.

Conclusion

Building a custom GraphRAG pipeline from scratch is not merely a preference—it is a technical necessity for conversational data where speaker attribution, claim modeling, and temporal narrative tracking are core requirements.

The evidence is clear across all research domains:

- **Schema-constrained extraction produces measurably higher precision** (98.8% with ODKE+) than open extraction (40% predicate errors)
- **Multi-pass pipelines outperform single-pass** by 16%+ in triple F1 (ECR), 18% on MINE (KGGen), and >20% in precision at scale (SAC-KG)
- **Custom entity resolution reduces duplication by 45%** via cascaded approaches
- **Assertion-transformed dialogue dramatically outperforms raw transcript retrieval** (LoCoMo benchmark)
- **No pre-built framework** addresses the fundamental structural differences between monologic text and dialogue (OKE-D formalization)

The architecture described here—speaker-aware chunking, multi-pass extraction with structured outputs, a reified claim ontology, hybrid vector-graph retrieval, Leiden community detection, and bi-temporal modeling inspired by Graphiti—transforms a podcast corpus from an opaque blob of text into a queryable knowledge structure where every claim traces back to a person, a moment,

and a context. For 300 episodes at **$15-45 total extraction cost**, this represents a practical, production-ready system that improves incrementally as new episodes arrive without requiring costly full rebuilds.

The most underappreciated insight: the real value of a podcast knowledge graph is not in the entities it contains but in the **claims network it creates**. When you can query "show me every prediction about Selic from 2023-2025, who made each prediction, whether they agreed or disagreed with each other, and which predictions turned out correct"—that is when the graph earns its complexity.

The most underexplored area remains Rook's OKE-D formalization—perspective-aware ontologies for multi-speaker knowledge extraction represent the frontier where this system can contribute novel methodology. The combination of ISO GQL standardization, Neo4j's MCP servers, and incremental Leiden algorithms provides a production-ready infrastructure stack that didn't exist 18 months ago.